CS6160 Complexity

Topic: Randomized Computation: BPP, RP, ZPP
Lecturer: Wei-Kai Lin

Date: Oct 07, 2026
Scriber: Jinye He

In this lecture we give Turing machines access to random coins. We define probabilistic Turing machines, look at polynomial identity testing as a problem where randomness seems to help, and define the randomized classes BPP (two-sided error), RP and coRP (one-sided error), and ZPP (zero error). The material follows [AB09, Chapter 7].

1 Probabilistic Turing Machines

Definition 1 (Probabilistic Turing machine [AB09, Chapter 7])**.** A **probabilistic Turing machine** (PTM) is similar to a Turing machine, but there are two transition functions (δ_0, δ_1). At each step, δ_0 is executed with probability $\frac{1}{2}$ and δ_1 is executed with probability $\frac{1}{2}$, independently of all previous steps.

The bits b are the **random coins** of the computation. Each coin is read only once: it selects the transition of a single step and is never looked at again. We say that M runs in time $T(n)$ if for every input x and every outcome of the coins, M halts within $T(|x|)$ steps.

Comparison: PTM, DTM, NDTM. How do PTMs compare with deterministic TMs (DTMs) and nondeterministic TMs (NDTMs)? Can they simulate each other?

- **Simulating a DTM by a PTM.** A DTM with transition function δ is a PTM with $\delta_0 = \delta_1 = \delta$ that simply ignores its coins.
- **Simulating a PTM by a DTM.** A DTM has no randomness, but it can go over *all* $r \in \{0,1\}^{T(n)}$: on input x, it runs $M(x;r)$ for every r and counts the accepting ones. This computes $\Pr[M(x)=1]$ exactly, in time $2^{T(n)} \cdot O(T(n)) = 2^{O(T(n))}$. So randomness can always be removed, but the naive simulation is exponentially slower.
- **PTM vs. NDTM.** Syntactically, a PTM is the same object as an NDTM: both have two transition functions (δ_0, δ_1), and every choice string $r \in \{0,1\}^{T(n)}$ determines one computation path. The difference is in how the tree of computation paths is interpreted. An NDTM accepts x iff *some* path accepts, i.e., $\exists\, r,\ M(x;r) = 1$, whereas for a PTM we care about the *fraction* of accepting paths, $\Pr_r[M(x;r)=1]$.

Example: ZEROP / PIT Before defining randomized complexity classes, we look at a natural problem for which randomness seems to help.

Definition 2 (Arithmetic circuit)**.** An **arithmetic circuit** C on variables $x_1, \ldots, x_m$ is similar to a Boolean circuit, but each wire carries an element of $\mathbb{Z}$ and each gate is one of $\{+, -, \times\}$ (with fan-in 2). Each input wire is a variable x_i (or a constant, such as 1). The circuit computes a polynomial $C(x_1, \ldots, x_m) \in \mathbb{Z}[x_1, \ldots, x_m]$ at its output. The **depth** of C is the length of the longest path from an input to the output.

Definition 3 (The problem ZEROP)**.**

$$ZEROP = \{\, \langle C \rangle : C \text{ is an arithmetic circuit and } C(x_1, \ldots, x_m) \equiv 0 \,\}.$$

That is, *ZEROP*(C) asks whether C is the zero polynomial: after expanding C into a sum of monomials, all coefficients vanish.

Remark 4. *ZEROP* captures Polynomial Identity Testing (PIT): two circuits C_1, C_2 compute the same polynomial iff the circuit $D(x) = C_1(x) - C_2(x)$ is in *ZEROP*.

How hard is *ZEROP*?

- The naive algorithm expands C into monomials and checks all coefficients. This can take exponential time, because a small circuit can have exponentially many monomials: the product $\prod_{i=1}^{m}(1 + x_i)$ is computed by a circuit of size $O(m)$, but

$$\prod_{i=1}^{m}(1 + x_i) = \sum_{S \subseteq [m]} \prod_{i \in S} x_i$$

  has 2^m terms.

- It is unclear whether *ZEROP* $\in \mathsf{P}$: no deterministic polynomial-time algorithm is known.

In contrast, there is a simple randomized algorithm, based on the following lemma.

Lemma 5 (Schwartz–Zippel [AB09, Chapter 7])**.** *Let $p(x_1, \ldots, x_m)$ be a polynomial over $\mathbb{F}_q$ with total degree d. If p is nonzero, then*

$$\Pr\big[p(a_1, \ldots, a_m) \neq 0\big] \;\geq\; 1 - \frac{d}{q},$$

where the probability is over the randomness of $a_1, \ldots, a_m \in \mathbb{F}_q$ (independent and uniform).

More generally, if p is a nonzero polynomial of total degree at most d over any field $\mathbb{F}$ (for instance $\mathbb{Q} \supseteq \mathbb{Z}$), and $a_1, \ldots, a_m$ are chosen independently and uniformly from a finite set $S \subseteq \mathbb{F}$, then $\Pr[p(a_1, \ldots, a_m) \neq 0] \geq 1 - d/|S|$.

The quantity d/q bounds the fraction of roots (solutions of $p = 0$) among all points. For $m = 1$ this is the familiar fact that a nonzero univariate polynomial of degree d has at most d roots. The lemma says that the same bound on the *fraction* of roots holds with many variables, even though a multivariate polynomial can have many more roots in absolute terms.

Randomized algorithm for *ZEROP*. On input an arithmetic circuit C with m variables and depth d:

1. Note that C computes a polynomial of degree at most 2^d: by induction on the depth, a $+$ or $-$ gate does not increase the degree beyond the maximum degree of its two inputs, while a $\times$ gate at most doubles it.
2. Randomly assign $a_i \leftarrow [10 \cdot 2^d] = \{1, 2, \ldots, 10 \cdot 2^d\}$, independently and uniformly for $i = 1, \ldots, m$, and evaluate $C(a_1, \ldots, a_m)$.
3. If $C(a_1, \ldots, a_m) = 0$, output "C is the zero polynomial"; otherwise, output "C is not the zero polynomial".

If $C \in$ *ZEROP*, then $C(a) = 0$ for every a, so the algorithm is always correct. If $C \notin$ *ZEROP*, then Lemma 5 with degree at most 2^d and $|S| = 10 \cdot 2^d$ (playing the role of q) gives

$$\Pr\big[C(a_1, \ldots, a_m) = 0\big] \;\leq\; \frac{2^d}{10 \cdot 2^d} \;=\; \frac{1}{10},$$

so the algorithm is correct with probability at least 9/10. Note that the error is *one-sided*: the algorithm can only err on circuits that compute a nonzero polynomial. This kind of error is formalized by the classes RP and coRP in Section 3.

2 The Classes BPTIME and BPP

Definition 6 (BPTIME and BPP [AB09, Chapter 7])**.** We say a PTM M **computes** a language L in time $T(n)$ if for all x, $M(x)$ halts within $T(|x|)$ steps and

$$\Pr\big[M(x) = L(x)\big] \;\geq\; \frac{2}{3},$$

where the probability is over the coins of the computation $M(x)$, and $L(x) \in \{0,1\}$ indicates whether $x \in L$.

- $\mathsf{BPTIME}(T(n))$ is the class of languages that can be computed by a PTM in $O(T(n))$ time.
- $\mathsf{BPP} = \bigcup_{c\geq 1} \mathsf{BPTIME}(n^c)$.

The "B" stands for *bounded* error: on every input, the error probability is at most 1/3. Note that the guarantee holds *for all* x, and that the running time is worst-case.

Clearly $\mathsf{P} \subseteq \mathsf{BPP}$. Moreover, the a DTM can try every possible random coins and tells whether $\Pr\big[M(x) = L(x)\big]$ is at least 2/3, so $\mathsf{BPTIME}(T(n)) \subseteq \mathsf{DTIME}(2^{O(T(n))})$. In particular, $\mathsf{BPP} \subseteq \mathsf{EXP}$.

2.1 Robustness of the Definition

The definition of BPP makes a few seemingly arbitrary choices: the constant 2/3, worst-case running time, and fair coins. We now check that none of them matters.

Constant greater than 1/2 defines the same class Replacing 2/3 by 3/4 gives the same class because we can repeat the computation k times independently and take the majority. If each run is correct with probability 2/3, then in expectation $(2/3)k$ of the k runs are correct, and the majority is wrong only if the number of correct runs drops to $k/2$ or below, far below its expectation. The Chernoff bound shows that this happens with probability only $2^{-\Omega(k)}$. But Replacing 2/3 by 1/2 makes the definition meaningless: a random guess satisfies $\Pr[M(x) = L(x)] = 1/2$ for *every* language L and the repetition trick doesn't work.

Theorem 7 (Chernoff bound [AB09, Appendix A])**.** *Let $X_1, X_2, \ldots, X_n$ be mutually independent random variables over $\{0,1\}$, and let $\mu = \sum_i \mathbb{E}[X_i]$. Then for all $c > 0$,*

$$\Pr\Big[\Big|\sum_i X_i - \mu\Big| \geq c\mu\Big] \;\leq\; 2 \cdot e^{-\mu \cdot \min\left(\frac{c^2}{4}, \frac{c}{2}\right)}.$$

Theorem 8 (Error reduction [AB09, Chapter 7])**.** *Let $0 < \epsilon \leq \frac{1}{2}$, and let M be a PTM that runs in time $T(n)$ such that $\Pr[M(x) = L(x)] \geq \frac{1}{2} + \epsilon$ for all x. For $k \geq 1$, let M_k be the PTM that, on input x, runs $M(x)$ for k times with independent coins and outputs the majority answer (say, 0 on a tie). Then M_k runs in time $O(k \cdot T(n))$, and for all x,*

$$\Pr\big[M_k(x) = L(x)\big] \;\geq\; 1 - 2e^{-\epsilon^2 k/8}.$$

Proof. Fix x, and let $X_i \in \{0,1\}$ indicate whether the i-th repetition outputs $L(x)$. The X_i's are mutually independent because the repetitions use independent coins, and $p := \mathbb{E}[X_i] = \Pr[M(x) = L(x)] \geq \frac{1}{2} + \epsilon$, so $\mu = pk$. The majority answer can be wrong only if $\sum_i X_i \leq k/2$, which means

$$\mu - \sum_i X_i \;\geq\; \mu - \frac{k}{2} \;=\; c\mu, \qquad \text{where } c := 1 - \frac{1}{2p}.$$

Since $p \geq \frac{1}{2}+\epsilon$ and $\epsilon \leq \frac{1}{2}$, we have $c \geq 1 - \frac{1}{1+2\epsilon} = \frac{2\epsilon}{1+2\epsilon} \geq \epsilon$. Moreover $c \leq \frac{1}{2}$, so $\min\left(\frac{c^2}{4}, \frac{c}{2}\right) = \frac{c^2}{4} \geq \frac{\epsilon^2}{4}$, and $\mu = pk \geq \frac{k}{2}$. By Theorem 7,

$$\Pr\Big[\Big|\sum_i X_i - \mu\Big| \geq c\mu\Big] \;\leq\; 2e^{-\mu c^2/4} \;\leq\; 2e^{-\frac{k}{2}\cdot\frac{\epsilon^2}{4}} \;=\; 2e^{-\epsilon^2 k/8}. \qquad \square$$

For the success probability $2/3$ in Definition 6, $p \geq 2/3$ gives $c \geq 1 - \frac{3}{4} = \frac{1}{4}$. This is the calculation from lecture: with $\mu = \frac{2}{3}k$ and $c = \frac{1}{4}$, whenever $\left|\sum_i X_i - \mu\right| < c\mu$ we have the majority is correct, and the failure probability is at most $2e^{-\frac{2k}{3}\cdot\frac{1}{64}} = 2e^{-k/96} = 2^{-\Omega(k)}$.

Consequences.

- *The constant does not matter.* If $\epsilon > 0$ is a constant, a constant number k of repetitions (any k with $2e^{-\epsilon^2 k/8} \leq \frac{1}{3}$) already gives success probability $2/3$; the same holds for $3/4$ or any other constant in $(\frac{1}{2}, 1)$. Since $O(k \cdot T(n)) = O(T(n))$, the classes $\mathsf{BPTIME}(T(n))$ and BPP are the same for all such constants.
- *Exponentially small error.* For $L \in \mathsf{BPP}$ and any polynomial $q(n)$, taking $k = \Theta(q(n))$ repetitions keeps the running time polynomial and reduces the error to $2^{-\Omega(q(n))}$.

Remark 9. The proof of Theorem 8 never used that ϵ is a constant. If $\epsilon = \epsilon(n) \geq 1/s(n)$ is only inverse-polynomial (for a polynomial s), then $k = 8\,s(n)^2 q(n)$ repetitions still give error at most $2e^{-q(n)}$ in polynomial time. In contrast, if we only require $\Pr[M(x) = L(x)] > \frac{1}{2}$ with no gap, the advantage over $\frac{1}{2}$ can be exponentially small and polynomially many repetitions no longer help.

Worst-case running time is enough Why do we require worst-case time T, not expected time T? Because it does not matter. Suppose a PTM M has expected running time at most $T(|x|)$ on every input x, for a time-constructible T, and satisfies $\Pr[M(x) = L(x)] \geq 2/3$. Let M' be the PTM that runs $M(x)$ for at most $10\,T(|x|)$ steps; if M has halted, M' outputs its answer, and otherwise M' outputs 0. By Markov's inequality,

$$\Pr\big[M(x) \text{ runs for more than } 10\,T(|x|) \text{ steps}\big] \;\leq\; \frac{1}{10},$$

so by a union bound $\Pr[M'(x) \neq L(x)] \leq \frac{1}{3} + \frac{1}{10} = \frac{13}{30}$, i.e., $\Pr[M'(x) = L(x)] \geq \frac{17}{30} = \frac{1}{2} + \frac{1}{15}$. Now M' runs in worst-case time $O(T(n))$, and repeating it a constant number of times brings the success probability back to $2/3$, still in worst-case time $O(T(n))$. In particular, defining BPP with expected polynomial time gives the same class.

Fair coins is enough Why not use "fancy" coins in a PTM, say Gaussian samples, or coins with an arbitrary bias? Fair coins turn out to be without loss of generality.

- A sample from a continuous distribution such as a Gaussian is a real number, which a TM cannot even write down exactly. Any computation only uses finitely many bits of precision, and good enough finite-precision approximations can be generated from fair coins.
- A coin that comes up 1 with probability ρ can be simulated with fair coins, using 2 tosses in expectation, as long as the bits of $\rho = 0.\rho_1\rho_2\rho_3\cdots$ (in binary) can be computed efficiently: toss fair coins $b_1, b_2, \ldots$, stop at the first i with $b_i \neq \rho_i$, and output 1 if $b_i < \rho_i$ (i.e., $b_i = 0$ and $\rho_i = 1$), and 0 otherwise. The first disagreement is at position i with probability 2^{-i}, so

$$\Pr[\text{output } 1] = \sum_{i:\, \rho_i = 1} 2^{-i} = \rho,$$

  and the expected number of tosses is $\sum_{i \geq 1} i \cdot 2^{-i} = 2$.

3 One-Sided Error: RTIME and RP

Definition 10 (RTIME and RP [AB09, Chapter 7])**.** $\mathsf{RTIME}(T(n))$ is the class of languages L such that there is a PTM M that runs in $O(T(n))$ time and, for all x,

$$\begin{aligned} x \in L: &\quad \Pr[M(x) = 1] \geq 2/3, \\ x \notin L: &\quad \Pr[M(x) = 0] = 1. \end{aligned}$$

$\mathsf{RP} = \bigcup_{c \geq 1} \mathsf{RTIME}(n^c)$. Also, $\mathsf{coRP} = \{L : \overline{L} \in \mathsf{RP}\}$.

An RP machine has *one-sided error*: it never accepts $x \notin L$, and it can only err on yes-instances, by rejecting. In particular, whenever it accepts, we know for sure that $x \in L$. Unfolding the definition, $L \in \mathsf{coRP}$ iff there is a polynomial-time PTM M (an RP machine for $\overline{L}$ with its output flipped) such that

$$\begin{aligned} x \in L: &\quad \Pr[M(x) = 1] = 1, \\ x \notin L: &\quad \Pr[M(x) = 0] \geq 2/3, \end{aligned}$$

i.e., M is perfect on yes-instances and can only err on no-instances.

Remark 11. For one-sided error, amplification is even simpler than for BPP: run M for k times and accept iff some run accepts. A no-instance is still never accepted, and a yes-instance is rejected with probability at most $(1/3)^k$. Hence the constant $2/3$ in the definition of RP can be replaced by any constant in $(0, 1)$, or even by $1/\mathrm{poly}(n)$.

ZEROP $\in \mathsf{coRP}$**.** The algorithm in Section 1 never errs on $C \in$ *ZEROP* and errs with probability at most $1/10$ on $C \notin$ *ZEROP*. Hence *ZEROP* $\in \mathsf{coRP}$. Equivalently, its complement (deciding whether a circuit computes a *nonzero* polynomial, or whether $C_1 \not\equiv C_2$) is in RP.

Proposition 12. $\mathsf{P} \subseteq \mathsf{RP} \subseteq \mathsf{NP}$ *and* $\mathsf{P} \subseteq \mathsf{coRP} \subseteq \mathsf{coNP}$.

Proof. $\mathsf{P} \subseteq \mathsf{RP}$: a polynomial-time DTM deciding L is a PTM (with $\delta_0 = \delta_1$) that is never wrong.

$\mathsf{RP} \subseteq \mathsf{NP}$: the random coins serve as an NP witness. Let M be an RP machine for L that runs in time $p(n)$, viewed as a deterministic machine $M(x; r)$ with $r \in \{0,1\}^{p(|x|)}$, and define the polynomial-time verifier $V(x, r) := M(x; r)$.

- If $x \in L$, then $\Pr_r[M(x;r) = 1] \geq 2/3 > 0$, so some r satisfies $V(x,r) = 1$.
- If $x \notin L$, then $\Pr_r[M(x;r) = 0] = 1$. Since each of the finitely many r has probability $2^{-p(|x|)} > 0$, we get $M(x;r) = 0$ for *every* r, so no witness exists.

Hence $x \in L \iff \exists r,\ |r| = p(|x|),\ V(x,r) = 1$, i.e., $L \in \mathsf{NP}$. The statements about coRP follow by taking complements. □

What about BPP? We have $\mathsf{P} \subseteq \mathsf{BPP}$, but we do not know whether $\mathsf{BPP} \subseteq \mathsf{NP}$. The argument above breaks down: for $x \notin L$, a BPP machine may still accept with probability up to $1/3$, so a no-instance can still have random coins r with $M(x;r) = 1$, and the existence of an accepting r no longer tells us anything.[1]

4 Zero Error: ZTIME and ZPP

Definition 13 (ZTIME and ZPP [AB09, Chapter 7])**.** $\mathsf{ZTIME}(T(n))$ is the class of languages L such that there is a PTM M that runs in *expected* time $O(T(n))$ and, for all x, if $M(x)$ halts, then $M(x) = L(x)$. $\mathsf{ZPP} = \bigcup_{c \geq 1} \mathsf{ZTIME}(n^c)$.[2] Also, $\mathsf{coZPP} = \{L : \overline{L} \in \mathsf{ZPP}\}$.

A ZPP machine never outputs a wrong answer; the randomness only affects its running time.

- Here the running time must be *expected*. With zero error and worst-case time $T(n)$ we would get nothing new: fixing all coins to 0 gives a DTM with the same running time that is always correct, so the class would just be $\mathsf{DTIME}(T(n))$.
- ZPP is closed under complement: swapping the outputs 0 and 1 of a zero-error machine for L gives a zero-error machine for $\overline{L}$ with the same running time. Hence $\mathsf{coZPP} = \mathsf{ZPP}$.

5 Relations Between the Classes

Theorem 14 ([AB09, Chapter 7])**.**

1. $\mathsf{ZPP} = \mathsf{RP} \cap \mathsf{coRP}$.
2. $\mathsf{RP} \subseteq \mathsf{BPP}$.
3. $\mathsf{coRP} \subseteq \mathsf{BPP}$.

Proof sketch of Theorem 14. Items 2 and 3 are immediate from the definitions: an RP machine M for L satisfies $\Pr[M(x) = 1] \geq 2/3$ for $x \in L$ and $\Pr[M(x) = 0] = 1 \geq 2/3$ for $x \notin L$, so $\Pr[M(x) = L(x)] \geq 2/3$ for all x, and $L \in \mathsf{BPP}$. The case of coRP is symmetric.

For item 1, we show both inclusions.

(i) $\mathsf{ZPP} \subseteq \mathsf{RP} \cap \mathsf{coRP}$. Let M be a zero-error PTM for L with expected running time at most $T(n) = \mathrm{poly}(n)$. Let M_0 run $M(x)$ for $3\,T(|x|)$ steps, output M's answer if M has halted, and output 0 otherwise. By Markov's inequality, M fails to halt within $3\,T(|x|)$ steps with probability at most $1/3$. If $x \notin L$, then M_0 always outputs 0 (either M's answer, which is correct, or the default 0). If $x \in L$, then M_0 outputs 1 whenever M halts in time, i.e., with probability at least

[1] It is known, however, that $\mathsf{BPP} \subseteq \Sigma_2^{\mathrm{p}} \cap \Pi_2^{\mathrm{p}}$ (Sipser–Gács–Lautemann); see [AB09, Chapter 7].

[2] The definition slide writes ZP; we use the name ZPP from textbook [AB09, Definition 7.7](zero-error probabilistic polynomial time).

2/3. So M_0 shows $L \in \mathsf{RP}$. The machine M_1 defined in the same way but with default output 1 shows $L \in \mathsf{coRP}$.

(ii) $\mathsf{RP} \cap \mathsf{coRP} \subseteq \mathsf{ZPP}$. Supposing that L is in both RP and coRP, we have polynomial-time PTMs A and B such that (a) A always outputs 0 for negative instances and (b) B always outputs 1 for positive instances. Thus $A(x) = 1$ certifies $x \in L$ and $B(x) = 0$ certifies $x \notin L$, so running both A and B gives zero error on both sides. Let $T_A(n)$ and $T_B(n)$ be the worst-case running times of A and B, both polynomial in n. We construct a PTM Z as follows.

On input x, Z repeats the following round until it halts:

1. Run $A(x)$ and $B(x)$ with fresh coins.
2. If $A(x) = 1$, halt and output 1. Else, if $B(x) = 0$, halt and output 0. Otherwise ($A(x) = 0$ and $B(x) = 1$), the round is inconclusive; go to the next round.

In particular, Z never stops after an inconclusive round, however unlucky its coins are. We claim that Z decides L with zero error and is expected to halt in polynomial time.

Zero error. If Z outputs 1, then $A(x) = 1$, so $x \in L$ by (a); if Z outputs 0, then $B(x) = 0$, so $x \notin L$ by (b). Hence whenever Z halts, $Z(x) = L(x)$.

Polynomial expected running time. Each round takes at most $T_A(n) + T_B(n)$ steps, where $n = |x|$. In each round, Z halts with probability at least 2/3: if $x \in L$, $A(x)$ outputs 1 with probability at least 2/3; if $x \notin L$, $B(x)$ outputs 0 with probability at least 2/3. So the expected number of rounds of Z is at most 3/2 and the expected running time is at most $\frac{3}{2}(T_A(n) + T_B(n))$, which is polynomial in n. Thus $L \in \mathsf{ZPP}$.

□

Summary. Putting everything together,

$$\mathsf{P} \subseteq \mathsf{ZPP} = \mathsf{RP} \cap \mathsf{coRP}, \qquad \mathsf{RP} \subseteq \mathsf{NP} \cap \mathsf{BPP}, \qquad \mathsf{coRP} \subseteq \mathsf{coNP} \cap \mathsf{BPP}, \qquad \mathsf{BPP} \subseteq \mathsf{EXP},$$

while it is open whether $\mathsf{BPP} \subseteq \mathsf{NP}$.

References

[AB09] Sanjeev Arora and Boaz Barak. *Computational Complexity: A Modern Approach*. Cambridge University Press, 2009.